

Reviewer Pack — Hypercontractive $p \rightarrow 2$ Norm of Restriction Operator Bounds Res...

Entry a0e6af1e7a28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 20:12:25 UTC

Hypercontractive $p \rightarrow 2$ Norm of Restriction Operator Bounds Resolution Width

Entry ID: a0e6af1e7a28 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 20:12:25 UTC

1. Conjecture

Field A (mathematical branch): Hypercontractivity / Bonami-Beckner semigroup $p \rightarrow 2$ norm estimates on the Boolean slice **Field B** (complexity object): Resolution refutation width $w(F \vdash \perp)$ of small unsatisfiable 3-CNF formulas

Statement:

Let F be an unsatisfiable 3-CNF on n variables, and let $A_F: \{0,1\}^n \rightarrow \{0,1\}$ be the assignment-acceptance indicator (1 iff a partial assignment violates no clause). Define $H(F) := \sup_{p \in [1,2]} \log(\|T_{\{1/\sqrt{p-1}\}} A_F\|_2 / \|A_F\|_p)$, the Bonami-Beckner-noise-amplified $p \rightarrow 2$ hypercontractive defect of A_F evaluated on a finite p -grid $\{1.1, 1.3, \dots, 1.9\}$. The conjecture: $w(F \vdash \perp) \geq \lceil H(F)/\log_2(n+1) \rceil$ for every unsatisfiable 3-CNF F with $n \leq 20$, with equality holding for at least 60% of the random unsatisfiable formulas at clause density 4.6.

Rationale (proposer's reasoning):

Hypercontractivity controls how mass concentrates on low-degree Fourier levels, and the Ben-Sasson-Wigderson width-degree correspondence relates resolution width to spectral concentration of the unsatisfiable region's indicator. A defect in the Bonami-Beckner inequality at small p quantifies high-degree Fourier energy of A_F , which should obstruct narrow resolution refutations: a narrow refutation forces A_F to be approximated by a low-degree multilinear polynomial. Unlike total-influence approaches, the $p \rightarrow 2$ norm probes the entire Fourier spectrum at multiple noise scales, capturing graded structure that a single influence number cannot.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9f1667702c6eacb5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 40 random unsat 3-CNFs per (n,density) cell with $n \in \{8,10,12,14\}$ and $\text{density} \in \{4.4, 4.6, 4.8, 5.0\}$ (640 instances total, fixed seeds 0-39), the inequality $w(F \vdash \perp) \geq \lceil H(F)/\log_2(n+1) \rceil$ must hold for every instance, and equality must hold for $\geq 60\%$ of the 160 instances at density 4.6.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.93	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - hypercontractivity resolution width lower bound 3-CNF refutation - Bonami-Beckner p-to-2 norm proof complexity unsatisfiable CNF - noise operator Boolean slice resolution refutation width random 3-SAT

Top relevant hits considered: - [http://arxiv.org/abs/1905.12372v1] Resolution Lower Bounds for Refutation Statements - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer - [http://arxiv.org/abs/0905.1587v5] Unsatisfiable Linear CNF Formulas Are Large and Complex - [http://arxiv.org/abs/0806.1148v2] Unsatisfiable CNF Formulas need many Conflicts - [http://arxiv.org/abs/0708.2336v1] Unsatisfiable Linear k-CNFs Exist, for every k - [http://arxiv.org/abs/1908.05361v2] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy - [http://arxiv.org/abs/0706.3351v2] Random Boolean Networks

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json

def generate_3cnf(n, density):
    clauses = []
    for _ in range(int(density * n * (n - 1) / 2)):
        clause = [random.randint(-n, n) for _ in range(3)]
        while len(set(clause)) != 3:
            clause = [random.randint(-n, n) for _ in range(3)]
        clauses.append(tuple(sorted(clause)))
    return clauses

def is_satisfiable(clauses):
    n = max(abs(c) for clause in clauses)
    assignments = list(itertools.product([0, 1], repeat=n))
```

```

for assignment in assignments:
    if all(any(x == y or x == -y for x, y in zip(assignment, clause)) for clause in clauses):
        return True
return False

def walsh_hadamard_transform(vector):
    n = len(vector)
    if n == 1:
        return vector
    even = walsh_hadamard_transform(vector[::2])
    odd = walsh_hadamard_transform(vector[1::2])
    result = [0] * n
    for i in range(n // 2):
        result[i] = even[i] + odd[i]
        result[i + n // 2] = even[i] - odd[i]
    return result

def hypercontractive_defect(A_F, p_values):
    H_F = []
    for p in p_values:
        T_p = [0] * len(A_F)
        for i in range(len(A_F)):
            T_p[i] = sum(A_F[j] * A_F[k] for j in range(i + 1) for k in range(j, len(A_F)) if (j -
            norm_2 = math.sqrt(sum(x**2 for x in T_p))
            norm_p = sum(abs(x)**p for x in A_F) ** (1 / p)
            H_F.append(math.log(norm_2 / norm_p))
    return max(H_F)

def resolution_width(clauses):
    n = max(abs(c) for clause in clauses)
    queue = list(range(n + 1))
    while queue:
        width = queue.pop(0)
        new_queue = []
        for i in range(width, n + 1):
            if any(all(x == y or x == -y for x, y in zip(clauses[j], clause)) for j in range(len(c
                new_queue.append(i + 1))
        if not new_queue:
            return width
        queue.extend(new_queue)
    return n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    p_values = [1.1, 1.3, 1.5, 1.7, 1.9]
    support_count = 0
    equality_count = 0
    counterexample = ""

    for n in {8, 10, 12, 14}:
        for density in {4.4, 4.6, 4.8, 5.0}:
            for _ in range(10):
                clauses = generate_3cnf(n, density)
                if is_satisfiable(clauses):
                    continue
                A_F = [1 if any(x == y or x == -y for x, y in zip(clause, assignment)) else 0 for

```

```

        H_F = hypercontractive_defect(A_F, p_values)
        w_F = resolution_width(clauses)
        if w_F >= math.ceil(H_F / math.log2(n + 1)):
            support_count += 1
            if density == 4.6:
                equality_count += 1

    conjecture_holds = support_count == 40 * 4 * 10
    equality_fraction = equality_count / (40 * 4 * 10)

    return {
        "metric_name": "resolution_width",
        "metric_value": w_F,
        "instances_tested": 40 * 4 * 10,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample if not conjecture_holds else ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30c405ba.py", line 109, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30c405ba.py", line 83, in run_trial
    if is_satisfiable(clauses):
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_30c405ba.py", line 28, in is_satisfiable
    n = max(abs(c) for clause in clauses)

```

